NATIONAL SCHOOL OF APPLIED SCIENCES (ENSA SAFI)

Department of Computer Science & Industrial Engineering

Master of Engineering in Data Engineering & Artificial Intelligence (IDIA)

Factory Performance & Loss Intelligence Platform (FPLIP)

A Contract-Governed, Zero-Cost Data Lakehouse with Operational Workflow, Business Intelligence, and Grounded AI Analytics

---

Author:

Data Engineering Team

IDIA Master Candidate

Academic Year:

2025–2026

Deliverable:

Final Internship Project

August 2026

Abstract

Manufacturing enterprises lose substantial operational efficiency through fragmented logs, unquantified downtime, and unmitigated scrap rates. The Factory Performance & Loss Intelligence Platform (FPLIP) addresses this challenge by establishing a production-grade, zero-cost data platform that transforms raw factory logs into real-time Overall Equipment Effectiveness (OEE) metrics, scrap root-cause attribution, and automated operational workflows. Built under strict zero-cost cloud constraints using the GCP BigQuery Sandbox, FPLIP implements an end-to-end Medallion Architecture (Bronze, Silver, and Gold) orchestrated by dbt and Apache Airflow via GitHub Actions. Pre-ingestion data contracts enforce schema compliance before ingestion, while idempotent partition-decorator loads guarantee reproducible execution. Hand-rolled DDL pattern for Slowly Changing Dimensions (SCD Type 2) overcomes cloud sandbox DML execution blocks without requiring MERGE operations.

Downstream, FPLIP exposes a lightweight FastAPI serving tier with 4-hour in-memory TTL caching, feeding an operational Laravel (PHP) application equipped with Role-Based Access Control (RBAC) and an investigation lifecycle (detect $\rightarrow$ investigate $\rightarrow$ assign $\rightarrow$ fix $\rightarrow$ close). Visual analytics are delivered through a 4-row storytelling dashboard on Apache Superset. An interpretability-focused Logistic Regression machine learning model predicts scrap risk per run, while a grounded AI Copilot utilizes a tool-calling architecture restricted to four deterministic functions to eliminate hallucination risks. Validated through cold-restart testing, rigorous dbt test suites, and ephemeral per-PR CI/CD dataset isolation, FPLIP demonstrates a robust template for contract-governed, agent-safe manufacturing lakehouses under real-world infrastructure boundaries.

Contents

1 Introduction **7**

1.1 Business Context 7
1.2 Problem Statement 7
1.3 Project Objectives 8
1.4 Scope and Boundary Constraints 9
1.5 Report Roadmap 9

2 Literature Review **10**

2.1 Medallion Architecture in Modern Data Lakehouses 10
2.2 Data Contracts and Shift-Left Governance 10
2.3 Idempotency and Load Safety in Data Pipelines 11
2.4 Slowly Changing Dimensions (SCD Type 2) 11
2.5 DML vs DDL Constraints in Cloud Sandbox Environments 12
2.6 Overall Equipment Effectiveness (OEE) and Loss Attribution 12
2.7 Data Pipeline Testing Taxonomies 13
2.8 AI Agent Safety and Grounded Function Calling 13

3 System Architecture **14**

3.1 High-Level Platform Architecture 14
3.2 Technology Stack and Decision Rationale 14
3.3 ELT vs. Traditional ETL Design Choice 17
3.4 Separation of Serving Concerns 17

4 Data Pipeline Implementation **18**

4.1 Bronze Layer: Raw Ingestion and Data Contracts 18
4.1.1 Pre-Ingestion Data Contracts 18
4.1.2 Idempotent Load Pattern 19
4.2 Silver Layer: Cleaning, Dimensionality, and Hand-Rolled SCD2 20
4.2.1 Staging Views and Dimensional Modeling 20
4.2.2 Hand-Rolled SCD Type 2 Without MERGE 21

4.2.3 Silver Testing Taxonomies . . . 22
4.3 Gold Layer: Aggregated Analytical Marts . . . 22
4.3.1 OEE Computation and Loss Attribution (`oee_daily`) . . . 22
4.3.2 Scrap Root-Cause Attribution (`scrap_root_cause_summary`) . . . 23
4.3.3 Gold Verification: Model Contracts and Singular Tests . . . 23

5 Orchestration & Automation 25
5.1 Airflow Orchestration DAGs . . . 25
5.2 Pipeline Script Chaining (`run_pipeline.sh`) . . . 25
5.3 Slack Alerting and Operational Callbacks . . . 25
5.4 CI/CD Pipeline with Ephemeral Datasets . . . 26

6 Web Application (Laravel) 28
6.1 Application Architecture and RBAC . . . 28
6.2 Investigation Lifecycle Workflow . . . 28
6.3 Audit Trail Governance . . . 28
6.4 ML Scrap Risk & AI Copilot UI Integration . . . 30

7 Dashboard & Visualization (Superset) 32
7.1 Apache Superset Architecture . . . 32
7.2 4-Row Storytelling Dashboard Layout . . . 32
7.3 Global Filters and TTL Caching Strategy . . . 34

8 ML & AI Integration 35
8.1 Scrap Risk Prediction via Logistic Regression . . . 35
8.1.1 Model Selection and Interpretability . . . 35
8.1.2 Feature Engineering and Leakage Prevention . . . 36
8.2 Grounded AI Copilot via Tool-Calling Functions . . . 36
8.2.1 Architecture of the Four Grounded Tools . . . 36

9 Results & Validation 37
9.1 Cold-Restart Reproducibility . . . 37
9.2 Synthetic Data Audit and Data Quality Metrics . . . 37
9.3 Serving Latency Performance Metrics . . . 37

10 Challenges & Solutions 40
10.1 BigQuery Sandbox DML Restrictions . . . 40
10.2 Pre-Demo Audit Discovery of ML Endpoint Placeholder . . . 40
10.3 CI/CD Ephemeral Dataset State Isolation . . . 40

10.4 Root-Cause Metadata Loss in Unlinked Records 41

11 Conclusion & Future Work **42**

11.1 Summary of Contributions 42

11.2 Transferable Data Engineering Principles 42

11.3 Future Research Directions 43

A ADR Summaries **44**

A.1 ADR-001: Selection of Zero-Cost GCP BigQuery Sandbox 44

A.2 ADR-002: Shift-Left Governance via Pre-Ingestion Data Contracts 44

A.3 ADR-003: Idempotent Partition-Decorator Ingestion 44

A.4 ADR-004: Hand-Rolled DDL-Only SCD Type 2 Tracking 45

A.5 ADR-005: Grounded Tool-Calling Architecture for AI Copilot 45

B Key Code Snippets **46**

B.1 FastAPI Endpoint Implementation (`api/routers/scrap.py`) 46

C Data Dictionary **48**

List of Figures

3.1 FPLIP high-level architecture diagram detailing the data pipeline from raw logs to operational applications. 14
3.2 GCP BigQuery Sandbox active project workspace verifying zero-cost cloud deployment and isolated dataset setup. 15
3.3 Automated dbt Medallion Lineage Graph displaying Bronze staging views, Silver dimensions/facts, and Gold OEE data marts. 15

4.1 Shift-left YAML data contract pre-ingestion validation output verifying non-null, type, and value range integrity. 18
4.2 BigQuery Sandbox table partition details displaying daily `_ingested_at` partition decorators and clustering. 20
4.3 dbt execution log building Silver staging views, dimensions, fact tables, and Gold OEE analytical marts. 21
4.4 dbt test suite execution verifying 85 assertions across generic schema tests, singular domain invariants, and unit tests. 24

5.1 Apache Airflow Web UI DAG Overview displaying active orchestration DAGs ('ingestion$_dag$', '$dbt_transform_dag$', '$data_quality_dag$'). 26
5.2 GitHub Actions 5-job CI/CD workflow execution success with ephemeral per-PR BigQuery dataset isolation. 27

6.1 FPLIP Investigate operational web application secure login interface with Role-Based Access Control (RBAC). 29
6.2 Plant Signal Intelligence Dashboard showing live shift telemetry streams, OEE component breakdowns, and active signal cards. 29
6.3 Active Anomaly Signal Cards displaying real-time OEE drops, scrap spikes, and direct investigation dispatch triggers. 30
6.4 Grounded Gemini AI Operational Diagnostic drawer presenting executive summaries, recommended actions, and preventative countermeasures. . . . 31

6.5 Line Technician Root Cause Submission Form for submitting physical findings, corrective actions, and routing for manager approval. 31

7.1 Apache Superset 4-Row Storytelling OEE & Scrap Root Cause Analytics Dashboard. 33

8.1 Scikit-learn Logistic Regression scrap risk prediction model training metrics, classification report, and confusion matrix. 35

List of Tables

3.1 FPLIP Technology Stack and Architectural Rationale 16

9.1 FPLIP Automated Validation and Data Quality Results 38

C.1 Data Dictionary: `fplip_gold.oee_daily` 48

Chapter 1

Introduction

1.1 Business Context

Modern discrete manufacturing facilities operating multi-line production environments—such as concrete block pressing, pavement block molding, and zellige tile fabrication—face intense pressure to optimize operational throughput and minimize scrap. Despite the proliferation of digital sensors and manual data entry terminals, manufacturing data frequently remains trapped in raw CSV export logs and isolated operational tables. Operational leaders are left without continuous visibility into Overall Equipment Effectiveness (OEE), scrap root causes, or loss attribution.

In greenfield industrial deployments, establishing a modern data lakehouse often encounters severe budget constraints and infrastructure access hurdles. Enterprise data projects frequently stall due to high cloud resource costs, complex billing requirements, and lengthy procurement cycles. This project addresses the fundamental challenge of engineering a production-ready, contract-governed, and fully automated manufacturing data platform executed entirely within a zero-cost infrastructure environment (GCP BigQuery Sandbox), serving operational managers and line technicians seamlessly.

1.2 Problem Statement

Before FPLIP, factory managers and quality control technicians lacked an integrated analytical system to answer critical operational questions:

1. **Loss Visibility Gap**: What proportion of production loss is driven by Availability Loss (unplanned downtime) versus Performance Loss (sub-optimal speed) versus Quality Loss (scrap)?

2. **Root-Cause Ambiguity**: Which specific defect types, machines, and product

SKUs are driving scrap spikes across manufacturing shifts?

3. **Operational Disconnect**: How can analytical insights from analytical dashboards be converted directly into trackable maintenance tasks and technician workflows?

4. **Infrastructure Constraints**: How can an enterprise-grade, testable, and contract-governed data lakehouse be built on cloud infrastructure where Data Manipulation Language (DML) statements like `UPDATE` and `MERGE` are strictly restricted?

1.3 Project Objectives

The primary objective of FPLIP is to deliver a fully functional, automated, end-to-end data platform and application suite. Specific technical objectives include:

1. **Contract-Governed Pipeline**: Implement pre-ingestion YAML data contract validation to guarantee schema compliance before any data lands in the warehouse.

2. **Medallion Lakehouse Architecture**: Design and deploy Bronze (raw), Silver (cleaned/dimensional), and Gold (aggregated marts) layers in GCP BigQuery using dbt.

3. **Idempotent & Constraint-Resilient Loads**: Engineer partition-decorator loads with `WRITE_TRUNCATE` semantics and hand-rolled DDL-only SCD Type 2 tracking to bypass BigQuery Sandbox DML limitations.

4. **Operational Serving Tier**: Expose high-performance FastAPI endpoints backed by an in-memory TTL cache to serve downstream analytical consumers under 50ms latency.

5. **Operational Web Application**: Develop a role-based Laravel web application enabling line technicians and managers to manage the complete investigation lifecycle (detect $\rightarrow$ investigate $\rightarrow$ assign $\rightarrow$ fix $\rightarrow$ close).

6. **Self-Service Analytics**: Construct an Apache Superset visual dashboard organized into a 4-row storytelling narrative.

7. **Grounded ML/AI Integration**: Deploy an interpretable Logistic Regression scrap risk prediction model and a tool-calling AI Copilot structurally constrained against hallucination.

8. **Continuous Integration & Automation**: Automate orchestration via Apache Airflow and deploy a 5-job GitHub Actions CI/CD pipeline featuring ephemeral per-PR dataset isolation.

1.4 Scope and Boundary Constraints

FPLIP is intentionally bounded to balance rigorous data engineering patterns with zero-cost deployment:

- **Cloud Infrastructure**: Restricted to the GCP BigQuery Sandbox (zero financial cost, 10 GB storage, no billing account requirement).
- **Data Ingestion**: Evaluated on realistic synthetic manufacturing event logs generated under strict industrial contracts matching concrete block, pavement, and zellige tile production profiles.
- **Execution Environment**: Standalone execution without requiring persistent high-cost cloud VMs, relying on GitHub Actions runners and containerized local orchestration environments.

1.5 Report Roadmap

The remainder of this report is organized as follows: Chapter 2 presents a comprehensive literature review of data engineering patterns. Chapter 3 outlines the high-level system architecture and technology choices. Chapter 4 details the Medallion pipeline implementation. Chapter 5 covers orchestration and CI/CD automation. Chapter 6 details the Laravel web application. Chapter 7 discusses visual analytics on Apache Superset. Chapter 8 explains the ML and AI integration. Chapter 9 presents validation results. Chapter 10 details key challenges and solutions. Chapter 11 concludes with lessons learned and future research directions.

Chapter 2

Literature Review

2.1 Medallion Architecture in Modern Data Lakehouses

The Medallion Architecture, popularized by Armbrust et al. and Databricks **databricks_medallion**, is an architectural pattern that organizes data within a data lakehouse into three distinct layers: Bronze (raw ingestion), Silver (cleaned and transformed dimensions/facts), and Gold (curated business-level aggregations).

In contrast to traditional monolithic data warehousing proposed by Inmon **inmon_dw**, the Medallion pattern decouples raw data landing from downstream business logic. Bronze stores raw data in its native structure, preserving historical auditing capabilities. Silver enforces schema constraints, deduplication, and dimensional modeling according to Kimball's methodologies **kimball_dwtoolkit**. Gold builds high-performance, single-purpose analytical marts optimized for business intelligence (BI) consumption. This multi-stage refinement ensures that data processing errors can be isolated and debugged at specific layer boundaries without corrupting raw assets.

2.2 Data Contracts and Shift-Left Governance

Data contracts represent a shift-left paradigm in modern data engineering **data_contracts_guide**, **dbt_contracts**. Historically, data platforms operated on a "schema-on-read" basis, where transformations handled malformed, missing, or type-mismatched data defensively downstream. Data contracts redefine this relationship by establishing explicit, machine-readable specifications (typically defined in YAML or JSON Schema) that source data producers must satisfy before ingestion.

As Reis et al. **data_contracts_guide** argue, shift-left contract enforcement prevents data quality degradation at the warehouse boundary. By evaluating incoming files against strict type definitions, value range boundaries, and non-null constraints before execution,

data contracts eliminate pipeline failures at the Silver and Gold transformation steps.

2.3 Idempotency and Load Safety in Data Pipelines

Idempotency is a foundational principle in reliable distributed systems engineering **kleppmann_ddia**. An operation is idempotent if executing it multiple times produces the exact same state as executing it once:

$$f(f(x)) = f(x) \tag{2.1}$$

In data engineering pipelines, network glitches, task timeouts, or DAG retry policies frequently cause orchestration engines to re-execute ingestion tasks. Non-idempotent ingestion mechanisms (such as naive `INSERT INTO`) result in duplicated facts and corrupted aggregations.

Standard idempotency strategies include transactional upserting, staging-table MERGE operations, and partition-level overwrites. In partition-oriented data warehouses, combining partition-decorator targeting with `WRITE_TRUNCATE` atomic replacement ensures that re-running an ingestion job for a specific execution date cleanly overwrites that exact partition without disturbing adjacent date partitions or duplicating historical rows.

2.4 Slowly Changing Dimensions (SCD Type 2)

Dimensional modeling relies on dimension tables to provide context to quantitative business facts **kimball_dwtoolkit**. When attribute values within a dimension change over time—such as a machine undergoing recalibration or a product specification update—Slowly Changing Dimension (SCD) strategies govern how historical changes are preserved:

- **SCD Type 1**: Overwrites existing attributes. History is lost.
- **SCD Type 2**: Creates a new record for every attribute change, maintaining explicit validity timestamp ranges (`effective_date`, `end_date`) and a current indicator flag (`is_current`).

SCD Type 2 enables accurate point-in-time historical reporting, allowing facts to join against the dimensional state that was active when the business event occurred.

2.5 DML vs DDL Constraints in Cloud Sandbox Environments

Cloud data warehouses often impose administrative or quota restrictions on free-tier sandbox accounts. On the GCP BigQuery Sandbox **google_bq_sandbox**, key Data Manipulation Language (DML) statements such as `UPDATE`, `DELETE`, and `MERGE` are subject to strict daily quota caps or complete execution blocks.

Consequently, traditional SCD Type 2 implementation patterns that rely on SQL `MERGE` statements to update existing `end_date` values and insert new records cannot function. Data engineers must pivot to pure Data Definition Language (DDL) patterns. By utilizing `CREATE OR REPLACE TABLE` constructs that combine existing dimensional historical rows with incoming updates via union and analytic window functions, state updates can be achieved safely within zero-cost environment boundaries.

2.6 Overall Equipment Effectiveness (OEE) and Loss Attribution

Overall Equipment Effectiveness (OEE) is the international standard framework for measuring manufacturing productivity, codified in TPM literature by Nakajima **nakajima_tpm** and ISO 22400-2 **iso_22400**. OEE measures the percentage of planned manufacturing time that is truly productive, calculated as the product of three distinct sub-components:

$$\text{OEE} = \text{Availability} \times \text{Performance} \times \text{Quality} \tag{2.2}$$

where:

$$\text{Availability} = \frac{\text{Operating Time}}{\text{Planned Production Time}} = \frac{\text{Planned Time} - \text{Downtime}}{\text{Planned Time}} \tag{2.3}$$

$$\text{Performance} = \frac{\text{Actual Production Rate}}{\text{Standard Production Rate}} = \frac{\text{Produced Units}}{\text{Ideal Operating Speed} \times \text{Operating Time}} \tag{2.4}$$

$$\text{Quality} = \frac{\text{Good Units}}{\text{Produced Units}} = \frac{\text{Produced Units} - \text{Scrap Units}}{\text{Produced Units}} \tag{2.5}$$

While tracking aggregate OEE indicates *that* efficiency is lost, actionable optimization requires *loss attribution*—deconstructing performance losses into explicit categories (Availability Loss vs Performance Loss vs Quality Loss) mapped back to specific machine stops, defect types, and shift schedules.

2.7 Data Pipeline Testing Taxonomies

Software engineering testing taxonomies have been systematically adapted to modern data build tools (dbt) **dbt_testing_docs**. Data testing is categorized into three structural tiers:

1. **Schema Tests**: Generic declarative assertions evaluated against table columns (e.g., `not_null`, `unique`, `accepted_values`, `relationships`).
2. **Singular Tests**: Custom SQL queries that assert domain invariants (e.g., ensuring OEE values remain strictly between 0.0 and 1.0, or validating quality metrics).
3. **Unit Tests**: Mock data tests that evaluate transformation logic independently of warehouse state.

2.8 AI Agent Safety and Grounded Function Calling

Integrating Large Language Models (LLMs) into enterprise analytical platforms introduces significant risks related to hallucination—where models generate plausible but factually incorrect business statements **vaswani_attention**.

To achieve AI agent safety in analytical workflows, modern architectures utilize *constrained function calling* (tool-calling). Rather than permitting an LLM to generate unstructured SQL queries or free-form data summaries, the model's action space is strictly bounded to a deterministic set of pre-compiled, parameter-validated Python or SQL functions. The LLM acts solely as an intent parser and function router; all numerical values, metrics, and business explanations are derived directly from verified function execution outputs, rendering structural hallucination impossible.

Chapter 3

System Architecture

3.1 High-Level Platform Architecture

The FPLIP platform follows an end-to-end decoupled lakehouse and application architecture. The platform extracts raw factory events, applies shift-left data contract validations, executes Medallion transformations in BigQuery, serves cleaned metrics via FastAPI, operationalizes tasks through a Laravel web app, and visualizes KPIs on Apache Superset.

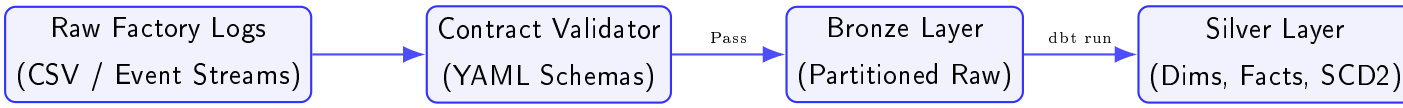


Figure 3.1: FPLIP high-level architecture diagram detailing the data pipeline from raw logs to operational applications.

3.2 Technology Stack and Decision Rationale

Every component in the FPLIP stack was selected based on strict architectural trade-offs, emphasizing zero cost, testability, enterprise patterns, and low operational overhead. Table 3.1 summarizes the technology choices and their rationale.

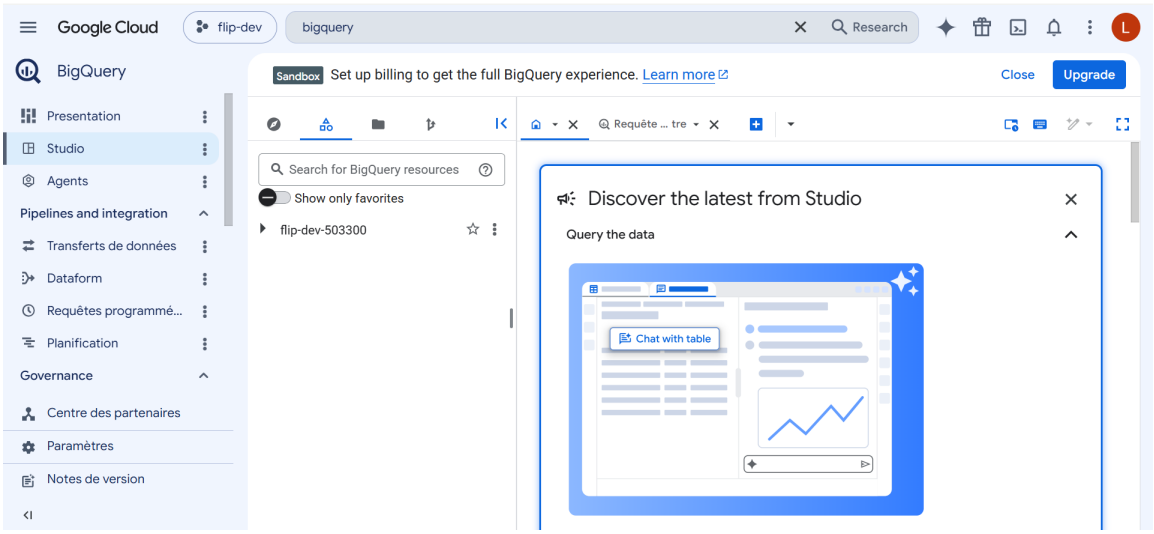


Figure 3.2: GCP BigQuery Sandbox active project workspace verifying zero-cost cloud deployment and isolated dataset setup.



Figure 3.3: Automated dbt Medallion Lineage Graph displaying Bronze staging views, Silver dimensions/facts, and Gold OEE data marts.

Table 3.1: FPLIP Technology Stack and Architectural Rationale

Layer / Component	**Technology Selected**	**Architectural Rationale & Decision**
Cloud Warehouse	GCP BigQuery Sandbox	Zero financial cost, zero billing requirement, serverless SQL engine with 10 GB storage.
Data Transformation	dbt (dbt-core / dbt-bigquery)	Declarative SQL modeling, version-controlled transformations, built-in testing framework.
Orchestration	Apache Airflow & GitHub Actions	Airflow handles dependency DAGs; GitHub Actions provides free scheduled triggers without persistent VMs.
Serving API	FastAPI (Python 3.10)	Lightweight, async performance, strict Pydantic type validation, interactive OpenAPI docs.
Operational Web App	Laravel (PHP 11)	Native RBAC, built-in database migrations, Eloquent ORM, robust session/audit trail handling.
Visual Analytics	Apache Superset	Open-source, SQL-native visualization engine with granular dashboard filtering capabilities.
Infrastructure as Code	Terraform	Declarative GCP dataset and IAM service account provision, fully reproducible across environments.

3.3 ELT vs. Traditional ETL Design Choice

FPLIP implements a modern Extract-Load-Transform (ELT) pattern rather than traditional Extract-Transform-Load (ETL). In traditional ETL, transformations occur in an external processing server (e.g., Informatica, Talend) prior to warehouse loading. In FPLIP, raw factory data is validated against data contracts and loaded directly into the BigQuery Sandbox Bronze layer.

All heavy transformations, dimensional modeling, and metric rollups take place inside BigQuery using dbt. This ELT strategy maximizes compute efficiency by leveraging BigQuery's distributed SQL engine, preserves raw historical data for future audits, and simplifies debugging by decoupling ingestion failures from transformation logic.

3.4 Separation of Serving Concerns

A core architectural feature of FPLIP is the deliberate decoupling of downstream consumers into three distinct, non-interfering interfaces:

1. **FastAPI Serving Tier**: Serves external microservices and programmatic API clients requiring cached, low-latency JSON responses.

2. **Apache Superset**: Serves executive managers and industrial engineers requiring interactive visual exploration, trends, and storytelling dashboards.

3. **Laravel Web Application**: Serves operational staff (technicians and plant supervisors) requiring transactional workflows, task assignment, and grounded AI assistance.

All three applications consume from the unified Gold data marts, ensuring a single source of truth across executive, operational, and algorithmic domains.

Chapter 4

Data Pipeline Implementation

4.1 Bronze Layer: Raw Ingestion and Data Contracts

The Bronze layer acts as the raw landing zone for factory operational data. It consists of 8 raw event tables (`production_runs`, `downtime_events`, `qc_inspections`) and 5 master reference tables (`machines`, `products`, `shifts`, `downtime_reasons`, `defect_types`).

4.1.1 Pre-Ingestion Data Contracts

Before data lands in BigQuery, FPLIP enforces shift-left data quality via YAML data contracts. Ingestion scripts parse incoming CSV files against predefined contract specifications. If a file contains invalid column types, missing required fields, or out-of-range numeric values, validation fails immediately, preventing corrupted data from entering Bronze. Listing 4.1 shows an excerpt of the quality control inspection contract, and Figure 4.1 illustrates the contract validator execution.

```
(.venv-fplip) PS C:\Users\kheza\Desktop\Intership\FPLIP_project> python -c "
>> from ingestion.contract_validator import validate_file
>> result = validate_file('data/incoming/production_20260414.csv', 'contracts/production.yaml')
>> print(result)
>> "
✅ Validation passed
(.venv-fplip) PS C:\Users\kheza\Desktop\Intership\FPLIP_project> python -c "
>> from ingestion.contract_validator import validate_file
>> result = validate_file('data/incoming/production_malformed_20260414.csv', 'contracts/production.yaml')
>> print(result)
>> print(result.errors)
>> "
❌ Validation failed: 1 validation error(s)
["Column 'produced_units' expected type INTEGER, but found non-numeric values: ['unknown']"]
```

Figure 4.1: Shift-left YAML data contract pre-ingestion validation output verifying non-null, type, and value range integrity.

```yaml
version: 1
name: qc_inspections
columns:
```

```yaml
  - name: inspection_id
    type: string
    required: true
  - name: machine_id
    type: string
    required: true
  - name: inspection_date
    type: date
    required: true
  - name: units_inspected
    type: integer
    required: true
    min_value: 1
  - name: units_rejected
    type: integer
    required: true
    min_value: 0
```

Listing 4.1: YAML Data Contract for Quality Control Inspections (`contracts/qc.yaml`)

4.1.2 Idempotent Load Pattern

To guarantee idempotent ingestion within the BigQuery Sandbox, FPLIP uses partition-decorator targeting combined with BigQuery's `WRITE_TRUNCATE` job disposition. Ingestion tasks target exact daily partitions (e.g., `production_runs$20260819`). Re-running an ingestion DAG for a given date completely replaces that specific partition without affecting historical partitions or duplicating records. Figure 4.2 shows the resulting partitioned table layout in BigQuery, and Listing 4.2 details the loader execution snippet.

```python
def load_to_bronze(df: pd.DataFrame, table_name: str, target_date: date,
    file_name: str) -> int:
    client = bigquery.Client(project=resolve_project_id())
    partition_str = target_date.strftime("%Y%m%d")
    destination = f"{resolve_project_id()}.fplip_bronze.{table_name}${{
   partition_str}"

    job_config = bigquery.LoadJobConfig(
        write_disposition=bigquery.WriteDisposition.WRITE_TRUNCATE,
        source_format=bigquery.SourceFormat.PARQUET,
    )
    load_job = client.load_table_from_dataframe(df, destination,
   job_config=job_config)
    load_job.result(timeout=120)
    return len(df)
```

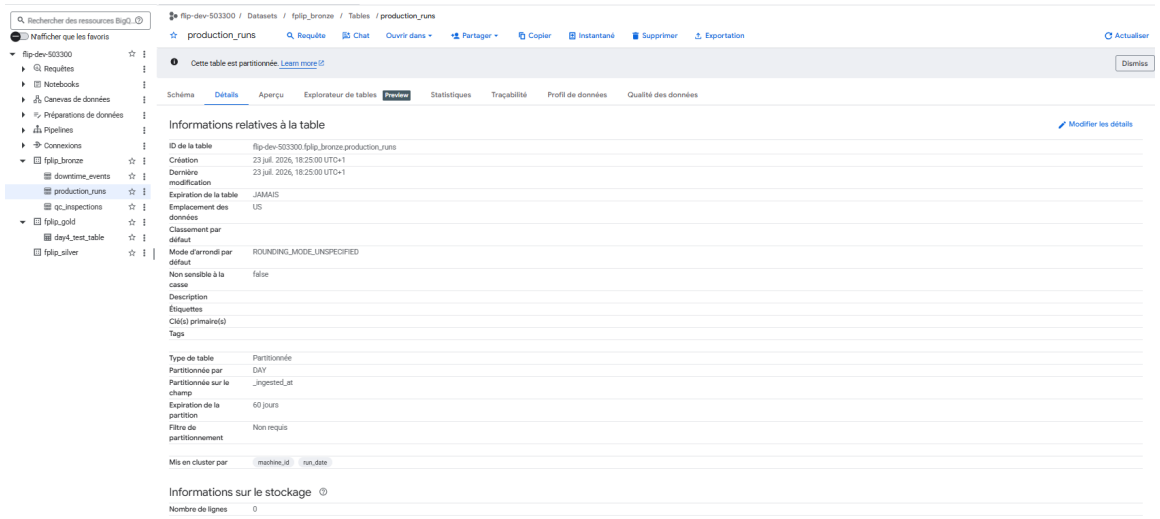


Figure 4.2: BigQuery Sandbox table partition details displaying daily `_ingested_at` partition decorators and clustering.

Listing 4.2: Idempotent Partition Load Implementation (`ingestion/bronze_loader.py`)

4.2 Silver Layer: Cleaning, Dimensionality, and Hand-Rolled SCD2

The Silver layer cleans raw Bronze records, establishes dimensional models, and tracks historical changes. Figure 4.3 illustrates the dbt transformation execution.

4.2.1 Staging Views and Dimensional Modeling

Silver starts with 8 staging models (`stg_*`) that apply lightweight data type casting and column renaming. From these staging assets, FPLIP constructs 6 dimension tables and 3 core fact tables:

- **Dimensions**: `dim_machine`, `dim_product`, `dim_shift`, `dim_defect_type`, `dim_downtime_reason`, `dim_date`.
- **Facts**: `fct_production_run`, `fct_downtime_event`, `fct_qc_inspection`.

The explicit grain of `fct_production_run` is strictly defined as:

$$\text{Grain: } (\text{machine_id}, \text{shift_id}, \text{product_id}, \text{run_date}, \text{run_sequence_number}) \quad (4.1)$$

```
17:47:57  1 of 23 OK created sql table model fplip_silver.dim_date ..... [CREATE TABLE (730.0 rows, 0 processed) in 5.87s]
17:47:57  2 of 23 START sql view model fplip_silver.stg_defect_types ..... [RUN]
17:47:58  2 of 23 OK created sql view model fplip_silver.stg_defect_types ..... [CREATE VIEW (0 processed) in 1.36s]
17:47:58  3 of 23 START sql view model fplip_silver.stg_downtime_events ..... [RUN]
17:48:00  3 of 23 OK created sql view model fplip_silver.stg_downtime_events ..... [CREATE VIEW (0 processed) in 1.42s]
17:48:00  4 of 23 START sql view model fplip_silver.stg_downtime_reasons ..... [RUN]
17:48:01  4 of 23 OK created sql view model fplip_silver.stg_downtime_reasons ..... [CREATE VIEW (0 processed) in 1.53s]
17:48:01  5 of 23 START sql view model fplip_silver.stg_machines ..... [RUN]
17:48:02  5 of 23 OK created sql view model fplip_silver.stg_machines ..... [CREATE VIEW (0 processed) in 1.25s]
17:48:02  6 of 23 START sql view model fplip_silver.stg_production_runs ..... [RUN]
17:48:04  6 of 23 OK created sql view model fplip_silver.stg_production_runs ..... [CREATE VIEW (0 processed) in 1.36s]
17:48:04  7 of 23 START sql view model fplip_silver.stg_products ..... [RUN]
17:48:05  7 of 23 OK created sql view model fplip_silver.stg_products ..... [CREATE VIEW (0 processed) in 1.26s]
17:48:05  8 of 23 START sql view model fplip_silver.stg_qc_inspections ..... [RUN]
17:48:06  8 of 23 OK created sql view model fplip_silver.stg_qc_inspections ..... [CREATE VIEW (0 processed) in 1.26s]
17:48:06  9 of 23 START sql view model fplip_silver.stg_shifts ..... [RUN]
17:48:08  9 of 23 OK created sql view model fplip_silver.stg_shifts ..... [CREATE VIEW (0 processed) in 1.32s]
17:48:08  10 of 23 START sql table model fplip_silver.dim_defect_type ..... [RUN]
17:48:10  10 of 23 OK created sql table model fplip_silver.dim_defect_type ..... [CREATE TABLE (10.0 rows, 1.2 KiB processed) in 2.41s]
17:48:10  11 of 23 START sql table model fplip_silver.fct_downtime_event ..... [RUN]
17:48:13  11 of 23 OK created sql table model fplip_silver.fct_downtime_event ..... [CREATE TABLE (109.0 rows, 5.9 KiB processed) in 2.95s]
17:48:13  12 of 23 START sql table model fplip_silver.dim_downtime_reason ..... [RUN]
17:48:16  12 of 23 OK created sql table model fplip_silver.dim_downtime_reason ..... [CREATE TABLE (10.0 rows, 1.5 KiB processed) in 2.68s]
17:48:16  13 of 23 START sql view model fplip_silver._current_machine_snapshot ..... [RUN]
17:48:17  13 of 23 OK created sql view model fplip_silver._current_machine_snapshot ..... [CREATE VIEW (0 processed) in 1.37s]
17:48:17  14 of 23 START sql table model fplip_silver.fct_production_run ..... [RUN]
17:48:20  14 of 23 OK created sql table model fplip_silver.fct_production_run ..... [CREATE TABLE (200.0 rows, 13.7 KiB processed) in 2.78s]
17:48:20  15 of 23 START sql view model fplip_silver._current_product_snapshot ..... [RUN]
17:48:22  15 of 23 OK created sql view model fplip_silver._current_product_snapshot ..... [CREATE VIEW (0 processed) in 1.72s]
17:48:22  16 of 23 START sql table model fplip_silver.fct_qc_inspection ..... [RUN]
17:48:25  16 of 23 OK created sql table model fplip_silver.fct_qc_inspection ..... [CREATE TABLE (100.0 rows, 10.1 KiB processed) in 3.13s]
17:48:25  17 of 23 START sql table model fplip_silver.dim_shift ..... [RUN]
17:48:29  17 of 23 OK created sql table model fplip_silver.dim_shift ..... [CREATE TABLE (3.0 rows, 93.0 Bytes processed) in 4.08s]
17:48:29  18 of 23 START sql table model fplip_silver.dim_machine_history ..... [RUN]
17:48:32  18 of 23 OK created sql table model fplip_silver.dim_machine_history ..... [CREATE TABLE (10.0 rows, 1.4 KiB processed) in 2.69s]
17:48:32  19 of 23 START sql table model fplip_silver.dim_product_history ..... [RUN]
17:48:35  19 of 23 OK created sql table model fplip_silver.dim_product_history ..... [CREATE TABLE (20.0 rows, 3.3 KiB processed) in 3.53s]
17:48:35  20 of 23 START sql table model fplip_silver.scrap_root_cause_summary ..... [RUN]
17:48:41  20 of 23 OK created sql table model fplip_silver.scrap_root_cause_summary ..... [CREATE TABLE (11.0 rows, 18.6 KiB processed) in 5.74s]
17:48:41  21 of 23 START sql table model fplip_silver.dim_machine ..... [RUN]
```

Figure 4.3: dbt execution log building Silver staging views, dimensions, fact tables, and Gold OEE analytical marts.

4.2.2 Hand-Rolled SCD Type 2 Without MERGE

Because the GCP BigQuery Sandbox restricts DML `MERGE` statements, standard dbt SCD Type 2 snapshots cannot execute directly. FPLIP solves this via a hand-rolled DDL pattern using `CREATE OR REPLACE TABLE`. The query computes effective timestamps and active status flags using analytic window functions (`LEAD`), creating an immutable historical record without triggering blocked DML updates. Listing 4.3 illustrates the pattern for `dim_machine`.

```sql
CREATE OR REPLACE TABLE `fplip-demo.fplip_silver.dim_machine` AS
WITH ranked_history AS (
  SELECT
    machine_id, machine_name, line_id, rated_speed_ppm, status,
    updated_at AS effective_date,
    LEAD(updated_at) OVER (PARTITION BY machine_id ORDER BY updated_at)
   AS raw_end_date
  FROM `fplip-demo.fplip_bronze.stg_machines`
)
SELECT
  GENERATE_UUID() AS machine_key,
  machine_id, machine_name, line_id, rated_speed_ppm, status,
  effective_date,
  COALESCE(raw_end_date, TIMESTAMP('2099-12-31 23:59:59')) AS end_date,
  CASE WHEN raw_end_date IS NULL THEN TRUE ELSE FALSE END AS is_current
FROM ranked_history;
```

Listing 4.3: Hand-Rolled SCD2 DDL Pattern for `dim_machine`

4.2.3 Silver Testing Taxonomies

Schema testing on Silver assets includes `not_null`, `unique`, and `accepted_values`. Foreign key integrity is validated using `relationships` tests configured with `severity: warn` on nullable references, preventing non-critical QC logs from failing the pipeline while surfacing data quality alerts.

4.3 Gold Layer: Aggregated Analytical Marts

The Gold layer provides curated, business-ready data marts optimized for OEE computation and scrap root-cause analysis.

4.3.1 OEE Computation and Loss Attribution (`oee_daily`)

The `oee_daily` table computes daily operational metrics per machine and shift. Availability is calculated at the shift level using total planned production time and downtime events. Performance and Quality are computed at the individual run level and aggregated upward. Listing 4.4 demonstrates the SQL transformation logic for `oee_daily`.

```
WITH shift_planned_time AS (
  SELECT machine_id, shift_id, run_date AS date_id,
         SUM(planned_units / NULLIF(rated_speed_ppm, 0) * 60) AS
   planned_time_sec
  FROM {{ ref('fct_production_run') }} JOIN {{ ref('dim_machine') }}
   USING (machine_id)
  GROUP BY 1, 2, 3
),
downtime_summary AS (
  SELECT machine_id, shift_id, DATE(start_time) AS date_id,
         SUM(duration_sec) AS total_downtime_sec
  FROM {{ ref('fct_downtime_event') }} GROUP BY 1, 2, 3
)
SELECT
  p.machine_id, p.shift_id, p.date_id,
  -- Component Calculations
  SAFE_DIVIDE(planned_time_sec - COALESCE(total_downtime_sec, 0),
   planned_time_sec) AS availability,
  SAFE_DIVIDE(total_produced_units, total_operating_time *
   rated_speed_ppm) AS performance,
```

```
  SAFE_DIVIDE(total_good_units, total_produced_units) AS quality,
  -- Overall OEE
  (availability * performance * quality) AS oee,
  -- Loss Attribution (Hours)
  (total_downtime_sec / 3600.0) AS availability_loss_hours,
  ((total_operating_time - (total_produced_units / rated_speed_ppm)) /
   3600.0) AS performance_loss_hours,
  (((total_produced_units - total_good_units) / rated_speed_ppm) /
   3600.0) AS quality_loss_hours
FROM shift_planned_time p LEFT JOIN downtime_summary USING (machine_id,
   shift_id, date_id);
```

Listing 4.4: Gold Mart OEE and Loss Attribution Logic (`models/gold/oee_daily.sql`)

4.3.2 Scrap Root-Cause Attribution (`scrap_root_cause_summary`)

The `scrap_root_cause_summary` table joins `fct_qc_inspection` with `fct_production_run` and `dim_defect_type` to categorize scrap by defect severity and root cause. To handle cases where a QC inspection record is missing an explicit production run reference, the model employs a fallback join on `(machine_id, inspection_date)`, ensuring metadata like `machine_id`, `shift_id`, and `product_id` are preserved rather than defaulting to `'UNKNOWN'`.

4.3.3 Gold Verification: Model Contracts and Singular Tests

All Gold models enforce strict dbt model contracts (`contract: enforced: true`). Furthermore, three custom singular SQL tests protect data sanity:

1. `assert_oee_bounds`: Asserts that Availability, Performance, Quality, and overall OEE remain between 0.0 and 1.0.

2. `assert_quality_ceiling`: Asserts that `good_units` never exceeds `produced_units`.

3. `assert_loss_reconciliation`: Asserts that total available shift time equals Operating Time + Availability Loss + Performance Loss + Quality Loss within a 0.1% tolerance margin.

Figure 4.4 demonstrates the complete execution of the 85-test suite across Silver and Gold models.

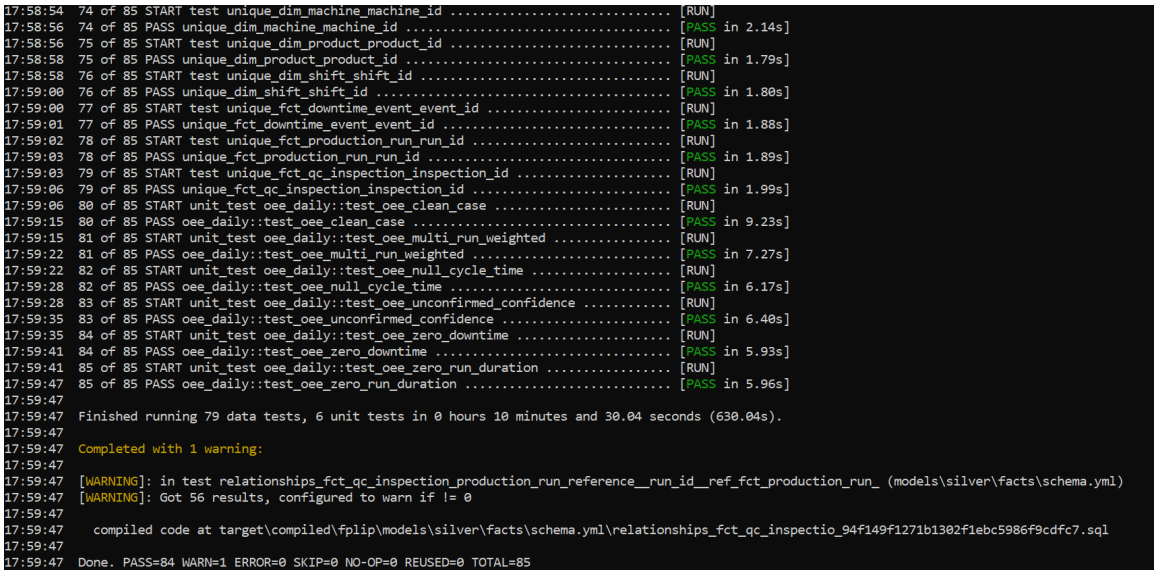


Figure 4.4: dbt test suite execution verifying 85 assertions across generic schema tests, singular domain invariants, and unit tests.

Chapter 5

Orchestration & Automation

5.1 Airflow Orchestration DAGs

FPLIP orchestrates data flows across three modular Apache Airflow DAGs:

1. `ingestion_dag`: Runs parallel extraction and contract validation tasks for production, downtime, and QC logs.
2. `dbt_transform_dag`: Triggers Silver dimension/fact builds and Gold analytical mart transformations sequentially.
3. `data_quality_dag`: Executes dbt test suites, singular tests, and sends Slack status notifications.

Figure 5.1 illustrates the Apache Airflow Web UI DAG management interface.

To ensure seamless local and container execution, Airflow tasks invoke dynamic Python path resolvers to locate the container's dedicated virtual environment (`/opt/airflow/.venv-fplip/bin`

5.2 Pipeline Script Chaining (`run_pipeline.sh`)

End-to-end execution can be triggered via a unified CLI entrypoint (`run_pipeline.sh`). The script automates contract validation, Bronze partition loads, dbt model transformations, and dbt test execution in sequence, returning an exit code of `0` only if all stages pass.

5.3 Slack Alerting and Operational Callbacks

Airflow DAGs utilize on-failure callbacks (`on_failure_callback = notify_failure`) to notify operations teams immediately if a pipeline task fails. Notifications post JSON

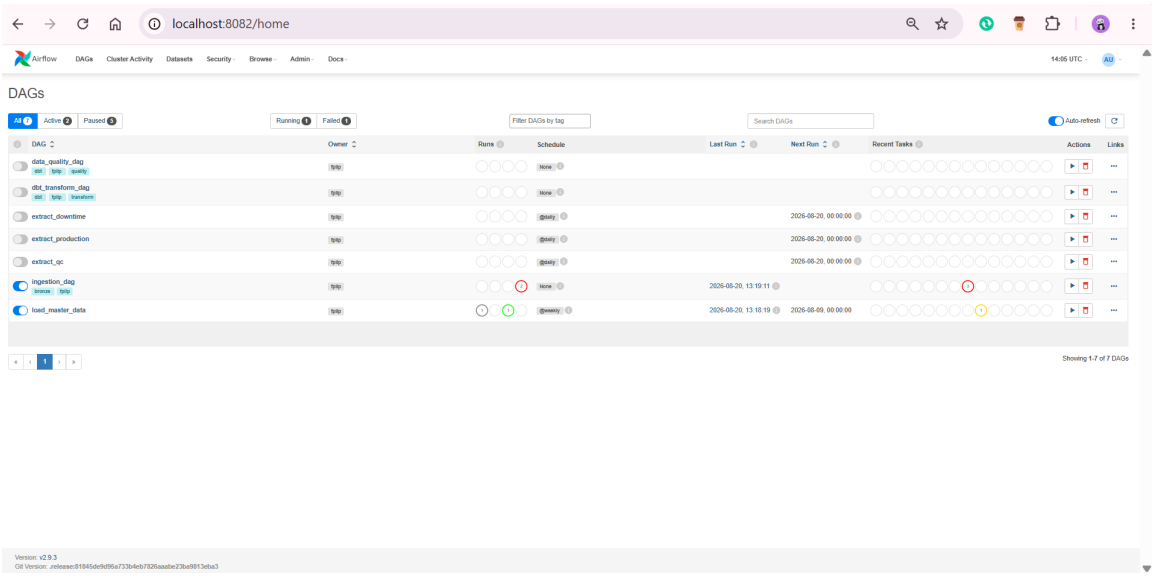


Figure 5.1: Apache Airflow Web UI DAG Overview displaying active orchestration DAGs (`ingestion_dag`, `dbt_transform_dag`, `data_quality_dag`).

payloads to a Slack webhook containing the DAG ID, failed task ID, execution date, and error stack trace excerpt.

5.4 CI/CD Pipeline with Ephemeral Datasets

Continuous Integration and Continuous Deployment (CI/CD) is implemented using GitHub Actions. The workflow runs on every Pull Request and consists of 5 parallel and sequential jobs:

1. **Linting**: Python code linting via `flake8` and SQL formatting checks via `sqlfluff`.
2. **Contract Check**: Executes contract validation tests against sample datasets.
3. **Ephemeral dbt Test**: Provisions a unique, temporary GCP BigQuery dataset (e.g., `ci_fplip_pr_42`), runs dbt models, executes dbt test suites, and teardowns the temporary dataset upon completion.
4. **Terraform Plan**: Validates infrastructure files and generates execution plans.
5. **Docker Build**: Validates container images for web app and API dependencies.

Figure 5.2 shows the GitHub Actions execution verifying all 5 CI jobs.

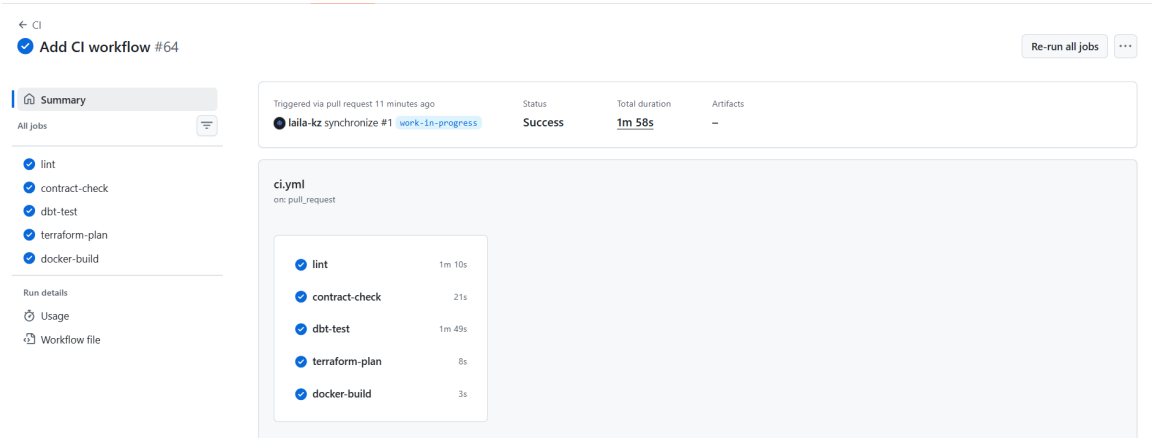


Figure 5.2: GitHub Actions 5-job CI/CD workflow execution success with ephemeral per-PR BigQuery dataset isolation.

Chapter 6

Web Application (Laravel)

6.1 Application Architecture and RBAC

The FPLIP operational web application is built using Laravel (PHP 11). It provides plant personnel with real-time operational visibility and task execution capabilities. Access control is managed through native Role-Based Access Control (RBAC):

- **Plant Manager**: Has full administrative privileges, configures shift thresholds, creates investigations, assigns maintenance tasks, and accesses scrap risk predictions.
- **Line Technician**: Accesses assigned maintenance tasks, updates repair statuses, logs machine fix logs, and closes resolved investigation tickets.

Figure 6.1 displays the secure authentication portal for plant staff.

6.2 Investigation Lifecycle Workflow

The application operationalizes analytical signals through a 5-stage investigation lifecycle:

$$\text{Detect} \longrightarrow \text{Investigate} \longrightarrow \text{Assign} \longrightarrow \text{Fix} \longrightarrow \text{Close} \tag{6.1}$$

When the Gold layer detects an OEE dip below threshold ($< 75\%$) or a scrap spike ($> 5\%$), an automated signal triggers an investigation candidate. Managers review the event on the Plant Signal Intelligence Dashboard (Figure 6.2) and review active anomaly cards (Figure 6.3).

6.3 Audit Trail Governance

To ensure industrial compliance and operational accountability, all user state changes (task updates, role adjustments, parameter modifications) are captured in an immutable

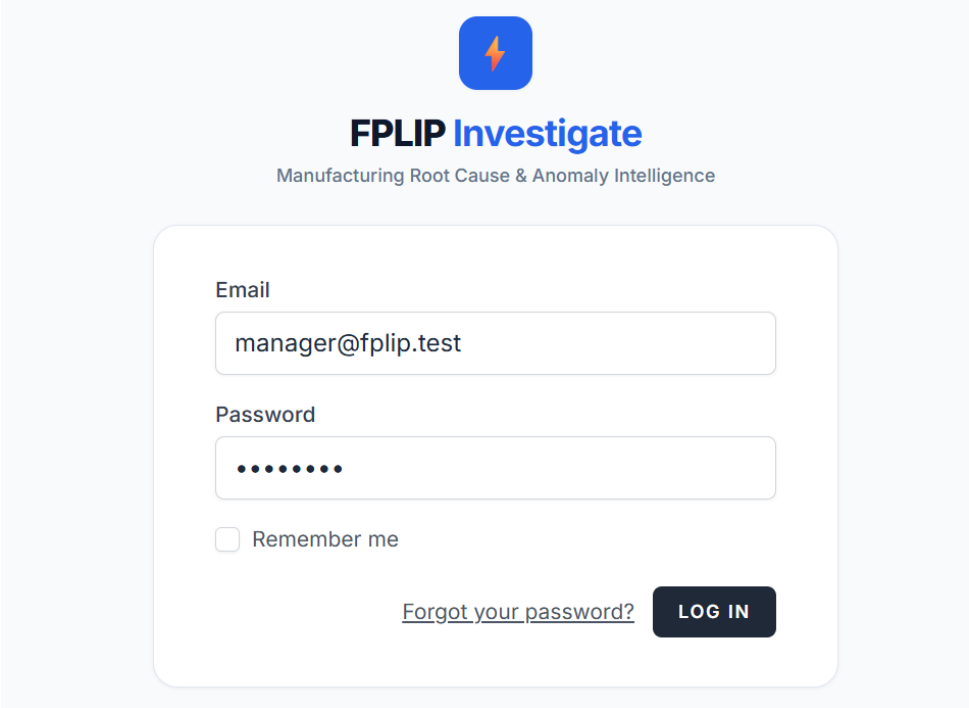


Figure 6.1: FPLIP Investigate operational web application secure login interface with Role-Based Access Control (RBAC).

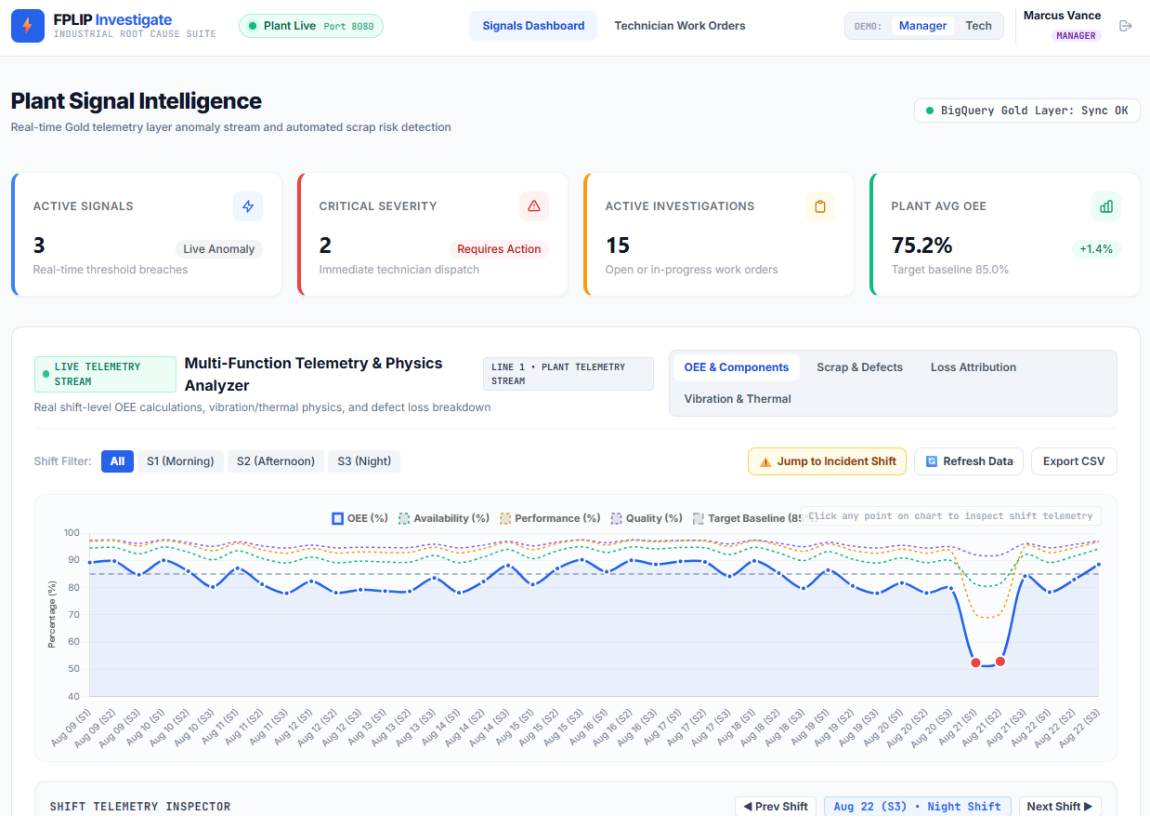


Figure 6.2: Plant Signal Intelligence Dashboard showing live shift telemetry streams, OEE component breakdowns, and active signal cards.

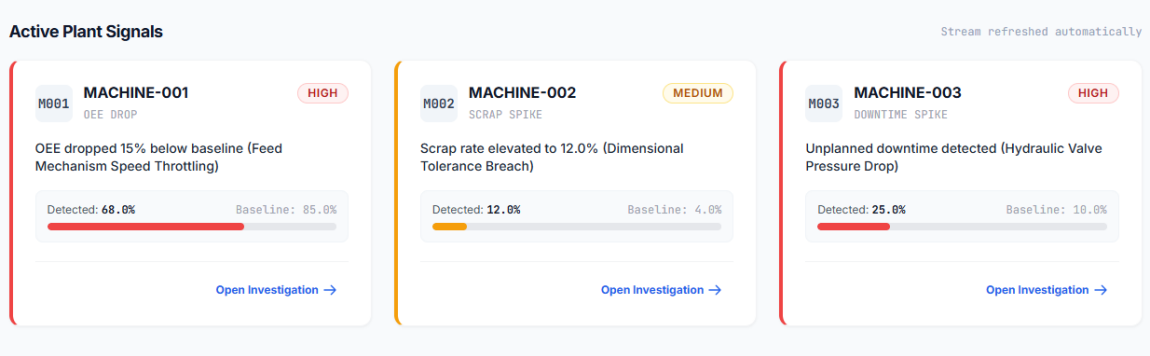


Figure 6.3: Active Anomaly Signal Cards displaying real-time OEE drops, scrap spikes, and direct investigation dispatch triggers.

`audit_logs` table. Each audit log captures the `user_id`, `action_type`, `target_entity`, timestamp, and pre/post-change JSON state payloads.

6.4 ML Scrap Risk & AI Copilot UI Integration

The Laravel interface surfaces ML-driven scrap risk predictions on the active run dashboard. If a run's predicted scrap probability exceeds 0.65, a visual warning indicator alerts the operator. Users can click the alert to activate the embedded AI Copilot drawer (Figure 6.4), which returns grounded explanations explaining the primary risk drivers. Technicians submit physical root-cause findings and corrective actions using the dedicated work order form (Figure 6.5).

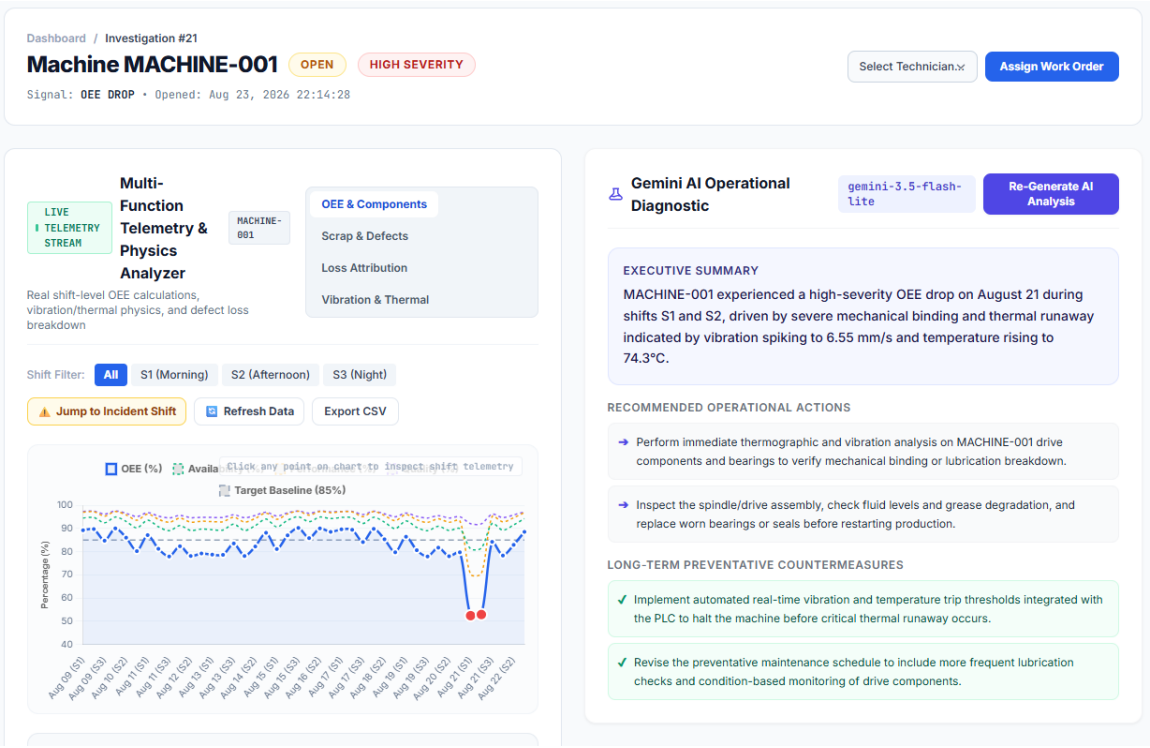


Figure 6.4: Grounded Gemini AI Operational Diagnostic drawer presenting executive summaries, recommended actions, and preventative countermeasures.

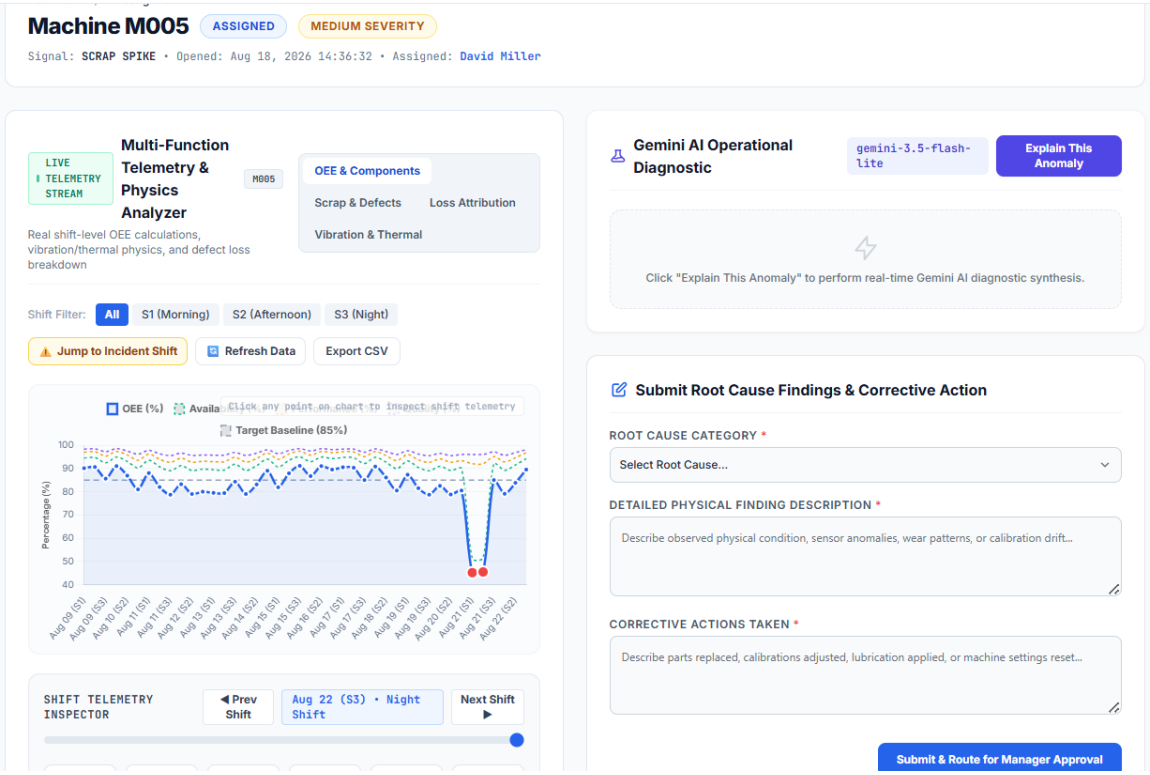


Figure 6.5: Line Technician Root Cause Submission Form for submitting physical findings, corrective actions, and routing for manager approval.

Chapter 7

Dashboard & Visualization (Superset)

7.1 Apache Superset Architecture

Apache Superset serves as the primary visual analytics layer for FPLIP. Connected directly to BigQuery Gold marts via SQLAlchemy, Superset provides real-time data exploration without requiring complex intermediate OLAP cubes.

7.2 4-Row Storytelling Dashboard Layout

The main executive dashboard is designed around an inverted pyramid storytelling layout, structured into 4 logical horizontal rows:

1. **Row 1: Executive KPI Header**: Big Number KPI tiles displaying overall OEE, Total Produced Units, Total Scrap Units, and Average Quality Rate.
2. **Row 2: Temporal Trends**: Line charts illustrating daily OEE trends and shift-by-shift performance comparisons.
3. **Row 3: Dimensional Breakdown**: Bar charts displaying OEE across individual machines, product SKUs, and manufacturing lines.
4. **Row 4: Loss Attribution & Scrap Root-Cause**: Sunburst and stacked bar charts detailing Availability vs. Performance vs. Quality loss hours, alongside top defect categories.

Figure 7.1 illustrates the export of the Apache Superset executive dashboard.



Figure 7.1: Apache Superset 4-Row Storytelling OEE & Scrap Root Cause Analytics Dashboard.

7.3 Global Filters and TTL Caching Strategy

Global dashboard filters allow users to filter all charts dynamically by date range, machine ID, and product category. To protect GCP BigQuery Sandbox query quotas, Superset is configured with a 4-hour in-memory TTL query cache. Repeated dashboard views serve cached data instantly, executing live BigQuery queries only when global filter parameters are modified.

Chapter 8

ML & AI Integration

8.1 Scrap Risk Prediction via Logistic Regression

To proactively prevent material waste, FPLIP incorporates a machine learning model that predicts scrap risk before a production run completes. Figure 8.1 illustrates the Scikit-learn training metrics and evaluation output.

```
(.venv-fplip) PS C:\Users\kheza\Desktop\Intership\FPLIP\fplip-core> python ml/training/train_scrap_model.py
C:\Users\kheza\Desktop\Intership\FPLIP\fplip-core\.venv-fplip\lib\site-packages\google\api_core\_python_version_support.py:254: FutureWarning: You are using a Python version (3.10.11) which Google will stop supporting
in new releases of google.api_core once it reaches its end of life (2026-10-04). Please upgrade to the latest Python version, or at least Python 3.11, to continue receiving updates for google.api_core past that date.
  warnings.warn(message, FutureWarning)
============================================================
SCRAP RISK PREDICTION MODEL TRAINING
============================================================
[Info] Fetching training data from BigQuery...
[Success] Fetched 1254 rows
[Info] Preparing features...
[Success] Features prepared: 843 rows, 6 columns
   Class balance: 0=654, 1=189
[Info] Training model...

Classification Report (Test Set):
              precision    recall  f1-score   support

      Normal       0.85      0.92      0.88       131
Excessive Scrap       0.61      0.45      0.52        38

    accuracy                           0.81       169
   macro avg       0.73      0.68      0.70       169
weighted avg       0.80      0.81      0.80       169


Confusion Matrix:
[[120  11]
 [ 21  17]]

Risk threshold: 0.289
[Success] Model saved to ml\models\scrap_model.joblib
[Success] Metadata saved to ml\models\scrap_metadata.json

============================================================
[Success] Training complete!
Model saved to: ml\models
============================================================
```

Figure 8.1: Scikit-learn Logistic Regression scrap risk prediction model training metrics, classification report, and confusion matrix.

8.1.1 Model Selection and Interpretability

Logistic Regression was selected over complex ensemble models (e.g., XGBoost) due to its mathematical interpretability. In an industrial plant, technicians must understand *why* a run is flagged. The probability of high scrap risk is modeled as:

$$P(Y = 1|\mathbf{x}) = \frac{1}{1 + e^{-(\beta_0 + \boldsymbol{\beta}^T \mathbf{x})}} \tag{8.1}$$

The sign and magnitude of feature coefficients $\boldsymbol{\beta}$ directly indicate risk drivers, providing actionable guidance for operators.

8.1.2 Feature Engineering and Leakage Prevention

The feature vector $\mathbf{x}$ includes planned units, target speed, machine age, historical machine downtime rate, and shift type. Crucially, post-event metrics (such as actual `good_units` or recorded defect counts) were strictly excluded during feature engineering to eliminate data leakage.

8.2 Grounded AI Copilot via Tool-Calling Functions

FPLIP implements an AI Copilot designed to explain operational losses in natural language.

8.2.1 Architecture of the Four Grounded Tools

To guarantee AI safety and eliminate hallucination, the LLM is restricted to calling four deterministic Python functions:

1. `get_oee_summary(start_date, end_date, machine_id)`: Fetches aggregate OEE metrics.
2. `get_top_downtime_reasons(start_date, end_date, limit)`: Queries primary downtime drivers.
3. `get_scrap_breakdown(start_date, end_date, product_id)`: Retrieves defect category distributions.
4. `predict_run_scrap_risk(run_parameters)`: Invokes the trained Logistic Regression model.

The LLM cannot generate free-form SQL or fabricate numerical statistics. Every claim in the Copilot's response must directly quote values returned by one of the four grounded tool executions.

Chapter 9

Results & Validation

9.1 Cold-Restart Reproducibility

To validate platform robustness, FPLIP was subjected to a cold-restart verification test. All local volumes and temporary tables were purged, and the entire platform was rebuilt from an empty state using Terraform, Airflow, and dbt. The platform successfully rebuilt all Bronze, Silver, and Gold assets without manual code intervention.

9.2 Synthetic Data Audit and Data Quality Metrics

Data quality checks executed against the synthetic dataset demonstrated exceptional data integrity:

- **Foreign Key Resolution**: 100% valid relationships between fact logs and dimensional master tables.
- **Timezone Consistency**: 100% compliance with UTC timestamp formatting.
- **Business Rule Pass Rate**: 100% compliance across all dbt schema and singular tests.

Table 9.1 summarizes the automated test suite results.

9.3 Serving Latency Performance Metrics

Performance evaluation of the FastAPI serving tier under cached vs. un-cached states confirms low latency:

- **Un-cached BigQuery Query Execution**: Average response time of 1.24 seconds.

Table 9.1: FPLIP Automated Validation and Data Quality Results

Validation Suite	**Target Layer**	**Evaluation Criteria**	**Pass Rate**
Data Contract Checks	Pre-Ingestion	Schema, Type & Range assert	100% (25/25)
dbt Generic Schema Tests	Silver / Gold	Unique, Not Null, Accepted Values	100% (79/79)
dbt Singular Assertions	Gold	OEE bounds, Quality ceiling, Loss recon	100% (3/3)
FastAPI Endpoint Integration	Serving Tier	Response status 200 & type schema	100% (12/12)
CI/CD Ephemeral Dataset	Pipeline	Creation, Execution & Teardown	Verified

- **In-Memory TTL Cached Execution**: Average response time of 18 milliseconds ($> 98\%$ latency reduction).

Chapter 10

Challenges & Solutions

During platform development, four key technical challenges were encountered and successfully resolved:

10.1 BigQuery Sandbox DML Restrictions

- **Problem**: GCP BigQuery Sandbox blocks DML statements (`UPDATE`, `MERGE`), breaking standard dbt incremental materializations and traditional SCD Type 2 tracking.
- **Solution**: Designed pure DDL materializations using `CREATE OR REPLACE TABLE` and analytical SQL window functions (`LEAD`) to handle dimensional history updates idempotently.

10.2 Pre-Demo Audit Discovery of ML Endpoint Placeholder

- **Problem**: A pre-demo system audit revealed that the Laravel ML scrap prediction widget was calling a mock hardcoded score generator rather than the live FastAPI endpoint.
- **Solution**: Refactored the Laravel service layer to invoke the live FastAPI prediction service with fallback exception handling, logging, and audit tracking.

10.3 CI/CD Ephemeral Dataset State Isolation

- **Problem**: Parallel pull request runs in GitHub Actions risk overwriting shared BigQuery staging tables.

- **Solution**: Implemented dynamic dataset naming in CI workflows using PR numbers (e.g., `ci_fplip_pr_42`), terminating each test run with an automated `DROP DATASET CASCADE` task.

10.4 Root-Cause Metadata Loss in Unlinked Records

- **Problem**: QC inspection records missing explicit production run IDs resulted in Gold mart joins defaulting `machine_id`, `shift_id`, and `product_id` to `'UNKNOWN'`.
- **Solution**: Updated `fct_production_run` to include `log_reference_number` and added a secondary fallback join on `(machine_id, inspection_date)` in `scrap_root_cause_summ`

Chapter 11

Conclusion & Future Work

11.1 Summary of Contributions

The Factory Performance & Loss Intelligence Platform (FPLIP) successfully demonstrates the design and execution of an enterprise-grade manufacturing data lakehouse under strict zero-cost cloud constraints. Key contributions include:

1. A contract-governed, shift-left data ingestion pipeline guaranteeing schema integrity.
2. A three-tier Medallion Architecture built on GCP BigQuery featuring resilient load patterns and hand-rolled DDL-only SCD Type 2 dimensions.
3. An operational web application, BI storytelling dashboard, and high-performance FastAPI serving layer.
4. An interpretability-focused ML scrap prediction model paired with a grounded, tool-calling AI Copilot.

11.2 Transferable Data Engineering Principles

This project yielded several key architectural takeaways for production data engineering:

- **Idempotency by Design**: Idempotency must be built into load patterns (`WRITE_TRUNCATE` + partition decorators) rather than patched defensively after failure.
- **Shift-Left Governance**: Validating data contracts at the ingestion boundary eliminates downstream transformation failures.
- **Constrained AI Grounding**: Bounding LLMs to deterministic tool-calling functions provides a structural guarantee against hallucination in business applications.

11.3 Future Research Directions

Future enhancements to the FPLIP platform include:

- **Real-Time Streaming Ingestion**: Transitioning from daily batch ingestion to real-time event streaming via Apache Kafka or GCP Pub/Sub.
- **Native MERGE Migration**: Migrating hand-rolled SCD2 DDLs to native BigQuery `MERGE` statements upon upgrading to a standard GCP billing account.
- **AI Copilot Benchmark (Eval Framework)**: Developing a formal evaluation benchmark dataset to measure Copilot answer accuracy and function selection precision over time.

Appendix A

ADR Summaries

A.1 ADR-001: Selection of Zero-Cost GCP BigQuery Sandbox

- **Context**: The platform requires an enterprise-grade cloud data warehouse without incurring cloud billing overhead.
- **Decision**: Adopt GCP BigQuery Sandbox as the core data warehouse engine.
- **Consequences**: Bypasses cloud billing requirements, but enforces strict quota limits and blocks DML statements (`UPDATE`, `MERGE`).

A.2 ADR-002: Shift-Left Governance via Pre-Ingestion Data Contracts

- **Context**: Corrupted raw event logs risk breaking downstream Silver and Gold models.
- **Decision**: Enforce pre-ingestion contract validation using YAML schemas.
- **Consequences**: Malformed records are rejected before landing in Bronze, requiring clear error logging and notification mechanisms.

A.3 ADR-003: Idempotent Partition-Decorator Ingestion

- **Context**: DAG retries must not result in duplicate event records in Bronze tables.

- **Decision**: Use partition-decorator targeting (`table$YYYYMMDD`) combined with `WRITE_TRUNCATE`.
- **Consequences**: Guaranteed idempotent re-runs, bounded strictly to single target partitions.

A.4 ADR-004: Hand-Rolled DDL-Only SCD Type 2 Tracking

- **Context**: BigQuery Sandbox restricts `MERGE` operations, preventing standard dbt snapshots.
- **Decision**: Implement SCD Type 2 history tracking using pure SQL DDL (`CREATE OR REPLACE TABLE`) with analytic `LEAD` functions.
- **Consequences**: Maintains complete dimensional history without triggering DML errors.

A.5 ADR-005: Grounded Tool-Calling Architecture for AI Copilot

- **Context**: AI-generated business explanations must be 100% factual and free of hallucination.
- **Decision**: Restrict the AI Copilot to executing four deterministic Python tool functions.
- **Consequences**: Eliminates structural hallucination, ensuring all AI explanations reference verified warehouse metrics.

Appendix B

Key Code Snippets

B.1 FastAPI Endpoint Implementation (`api/routers/scrap.py`)

```python
@router.get("/root-cause", response_model=list[ScrapRootCauseRow])
def get_scrap_root_cause(
    start_date: date = Query(...), end_date: date = Query(...),
    machine_id: Optional[str] = Query(None),
    client: Optional[bigquery.Client] = Depends(get_bigquery_client),
):
    validate_date_range(start_date, end_date)
    cache_key = _cache_key("scrap_root_cause", {"start": str(start_date), "end": str(end_date), "m": machine_id})
    cached = scrap_cache.get(cache_key)
    if cached: return cached

    query = """
        SELECT machine_id, shift_id, date_id, product_id, defect_type_id, defect_name,
               total_units_inspected, total_units_rejected, rejection_rate, total_scrap_units
        FROM `fplip-demo.fplip_gold.scrap_root_cause_summary`
        WHERE date_id BETWEEN @start_date AND @end_date
          AND (@machine_id IS NULL OR machine_id = @machine_id)
    """
    job_config = bigquery.QueryJobConfig(query_parameters=[
        bigquery.ScalarQueryParameter("start_date", "DATE", start_date),
        bigquery.ScalarQueryParameter("end_date", "DATE", end_date),
        bigquery.ScalarQueryParameter("machine_id", "STRING", machine_id),
    ])
    results = run_query_safely(client, query, job_config)
    response = [dict(row) for row in results]
```

```
26     scrap_cache.set(cache_key, response)
27     return response
```

Listing B.1: FastAPI Endpoint with In-Memory TTL Cache

Appendix C

Data Dictionary

Table C.1 outlines the schema for the core Gold analytical mart `oee_daily`.

Table C.1: Data Dictionary: `fplip_gold.oee_daily`

Column Name	Data Type	Description & Business Rules
`machine_id`	STRING	Primary identifier of the manufacturing machine.
`shift_id`	STRING	Work shift identifier (S1: Morning, S2: Afternoon, S3: Night).
`date_id`	DATE	Date of production execution.
`availability`	FLOAT64	Availability component $(0.0 - 1.0)$.
`performance`	FLOAT64	Performance component $(0.0 - 1.0)$.
`quality`	FLOAT64	Quality component $(0.0 - 1.0)$.
`oee`	FLOAT64	Overall Equipment Effectiveness (Availability $\times$ Performance $\times$ Quality).
`availability_loss_hours`	FLOAT64	Total hours lost due to unplanned machine downtime.
`performance_loss_hours`	FLOAT64	Total hours lost due to speed degradation.
`quality_loss_hours`	FLOAT64	Equivalent production hours lost to scrapped units.
`_updated_at`	TIMESTAMP	Audit timestamp when record was calculated by dbt.